

CRC Receiver Functional Design Specifications

CSI-2 Controller IP

Version 4.0.1

Block Owner

Mina Nasser

Authors

Mina Nasser

Contents

1	Revision History	3
2	Overview	4
3	Operation and Description	4
3.1	Digital Interface	4
3.1.1	Parameters Names	4
3.1.2	Ports Names	4
3.2	Functional Description	5
3.3	Block Diagram	6
3.4	Timing Diagram	6
3.5	Verification Requirements	7

1 Revision History

The following table:

Version	Date	Author(s)	Revision Notes	Owner Approval
1	2025-12-14	Mina Nasser	Initial FDS creation for crc_rx module.	

2 Overview

The `crc_rx` (CRC Receiver) block is a digital logic module responsible for verifying data integrity in the MIPI CSI-2 transmission protocol.

The module receives a continuous stream of 8-bit payload data bytes, followed by a 16-bit checksum transmitted as two bytes: Least Significant Byte (LSB) first, then Most Significant Byte (MSB). During payload reception, the `crc_rx` block computes a *reflected CRC-16* value in real time.

After receiving the final checksum byte (MSB), the internally computed CRC value is immediately compared with the received checksum. Based on this comparison, the module asserts a *Pass* signal if the packet is valid, or a *Fail* signal if data corruption is detected.

The CRC validation result is generated with zero additional latency, ensuring immediate packet integrity feedback.

3 Operation and Description

3.1 Digital Interface

3.1.1 Parameters Names

There are no configurable parameters exposed in the module interface.

3.1.2 Ports Names

The following table:

Port Name	Width	Type	Description
<code>clk</code>	1	Input	System Clock. Trigger on rising edge.
<code>reset_n</code>	1	Input	Active-low async reset. Resets CRC to 0xFFFF.
<code>i_data_valid</code>	1	Input	Indicates <code>i_data_in</code> is valid data.
<code>i_data_in</code>	8	Input	Input stream (Payload → LSB → MSB).
<code>i_start</code>	1	Input	Start of Packet (Sync Start with 1st byte).
<code>i_end</code>	1	Input	End of Payload Flag (Marks last data byte).
<code>o_crc_ok</code>	1	Output	High if CRC matches (Pass). Valid when Done is High.
<code>o_crc_done</code>	1	Output	1-cycle pulse indicating check completion.

3.2 Functional Description

The `crc_rx` block implements CRC-16 verification logic controlled by an internal Finite State Machine (FSM). The operation is divided into three main phases:

A. Initialization Phase

When the `i_start` signal is asserted, the internal CRC register (`crc_reg`) is forced to the initialization value `0xFFFF`. If `i_data_valid` is also asserted in the same clock cycle, the module simultaneously initializes the CRC state and processes the first incoming payload byte. This behavior enables a *synchronous start* without introducing additional latency.

B. Data Processing Phase

For each valid payload byte, when `i_data_valid` = 1 and the FSM is in the *Payload* state, the module performs a *reflected CRC update* as follows:

- The current 16-bit CRC state is XORed with the incoming 8-bit data byte.
- The intermediate value propagates through a combinational chain consisting of eight XOR-and-shift stages.
- The resulting CRC value is stored back into the `crc_reg` register on the next rising clock edge.

This process is repeated for every payload byte in the packet.

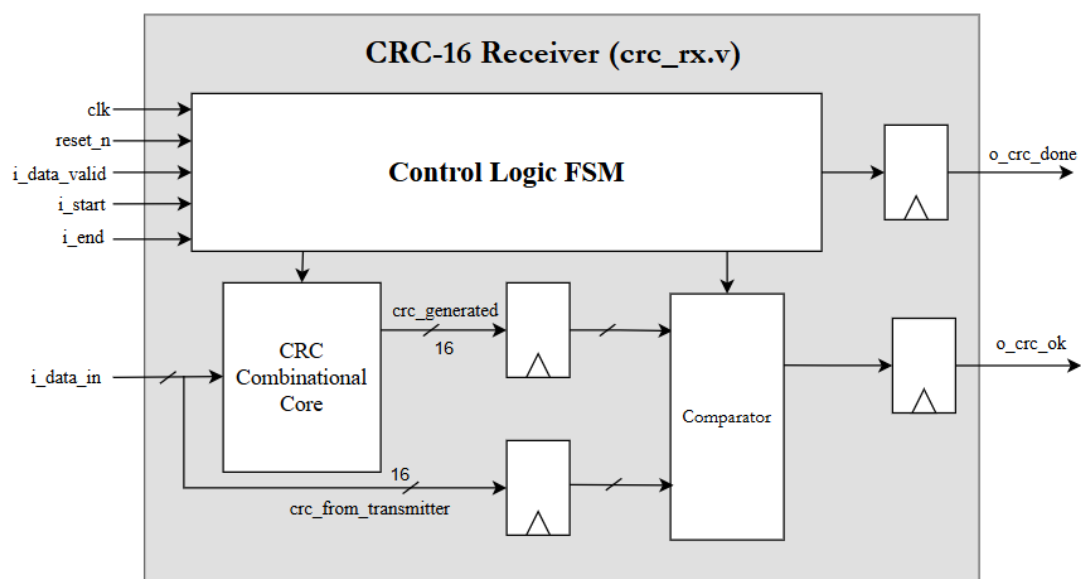
C. Check Phase

To achieve zero-latency CRC verification, the checksum comparison is performed without waiting for an additional clock cycle:

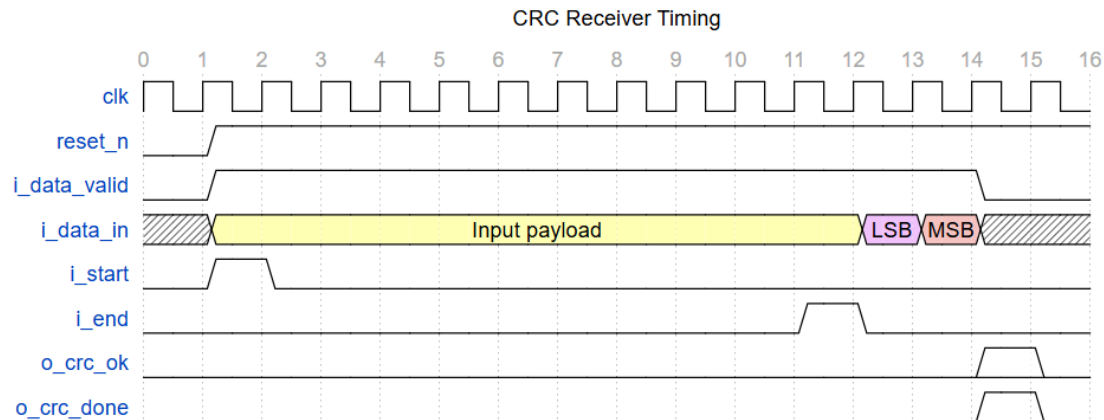
- **Cycle N (Last Payload):** The `i_end` signal is asserted, indicating the final payload byte. The FSM transitions to the state awaiting the checksum LSB.
- **Cycle $N+1$ (Receive LSB):** The first checksum byte (LSB) is received and stored in a temporary register (`rx_lsb`).
- **Cycle $N+2$ (Receive MSB and Compare):** Upon arrival of the checksum MSB, the module does not wait to store the byte. Instead, it forms a 16-bit checksum vector `{i_data_in, rx_lsb}` and immediately compares it, using combinational logic, against the internally computed `crc_reg`.

As a result, the signals `o_crc_done` and `o_crc_ok` are asserted in the same cycle as the MSB reception, providing immediate CRC validation with zero additional latency.

3.3 Block Diagram



3.4 Timing Diagram



3.5 Verification Requirements

The functional and timing behavior of the `crc_rx` module is verified using the `tb_crc_rx` testbench. The following requirements define the expected behavior:

- VR-1: Reset Functionality** The DUT shall enter a known idle state when `reset_n` is asserted. The outputs `o_crc_ok` and `o_crc_done` must be driven Low immediately.
- VR-2: Synchronous Start** The DUT shall correctly process the first payload byte if `i_start` and `i_data_valid` are asserted in the same clock cycle.
- VR-3: Payload Consumption** The DUT shall process exactly one byte per clock cycle while `i_data_valid` is High.
- VR-4: Checksum Capture** The DUT shall correctly interpret the two bytes following the `i_end` flag as the CRC LSB and CRC MSB, respectively.
- VR-5: Zero-Latency Verification** The DUT shall perform the CRC comparison and assert `o_crc_done` in the same clock cycle that the MSB is received. No idle cycles are allowed between the MSB reception and the Done signal.
- VR-6: Pass Condition** The output `o_crc_ok` shall be High if and only if the calculated CRC matches the received checksum.
- VR-7: Fail Condition** The output `o_crc_ok` shall be Low if the calculated CRC does not match the received checksum (e.g., due to a bit error in the payload).
- VR-8: Multiple Packet Support** The DUT shall verify multiple back-to-back packets correctly without requiring a hard reset between them.